Coronel, Joshua S.
C204

Finals Task 2: Inheritance

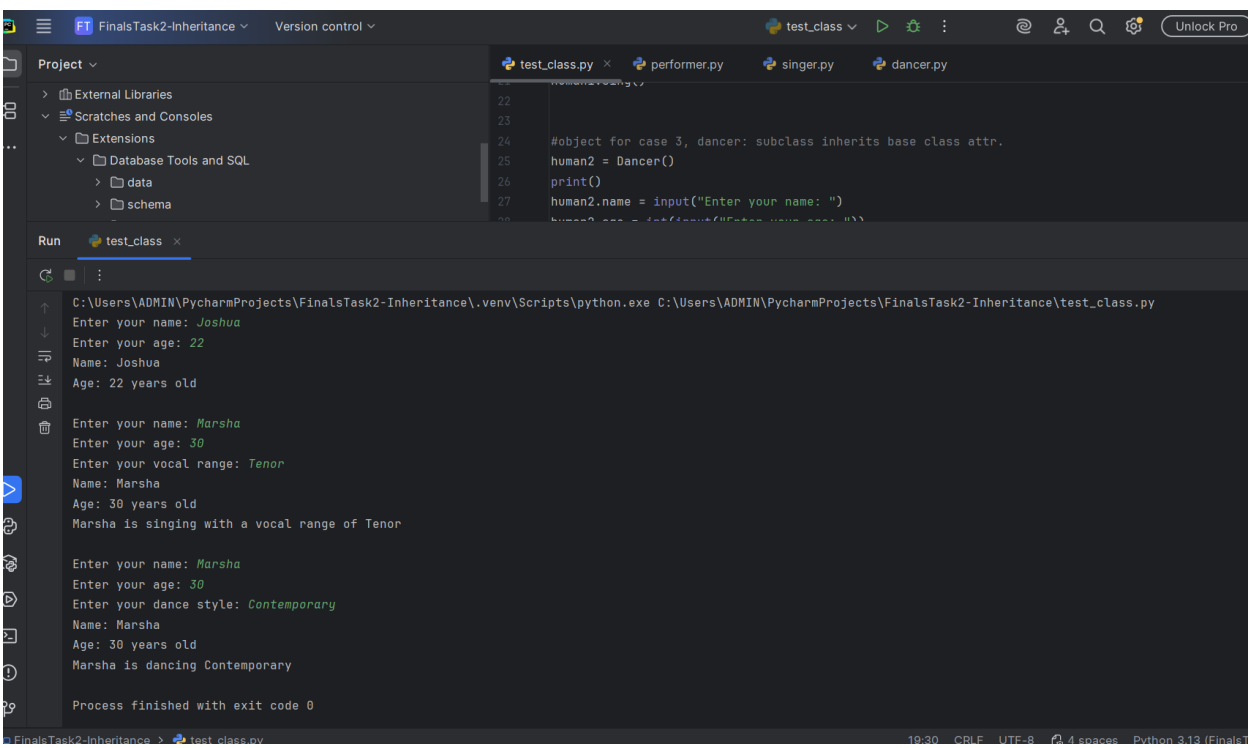
Problem School Performance
Note: You are to create 4 separate python files for this task:

- performer.py(base class)
- singer.py(sub class)
- dancer.py(sub class)
- test_class.py – following the required test cases

- in school music performance, different types of performer participates: for this program we will be implementing the performers:

Base Class Performer:
Sub Class Singer: inherits from Performer, with additional property "vocal_range"
: getter vocal_range, and method sing()
Sub Class Dancer: inherits from Performer, with additional property "Dance_Style"
:getter dance style, and method dance

Output:



Code:

Testclass:

```python
from performer import Performer
from singer import Singer
from dancer import Dancer

#object for case 1, base class
human0 =  Performer()
human0.name = input("Enter your name: ")
human0.age = int(input("Enter your age: "))
human0.info()  #base class Performer method prints performer "info()"

#object for case 2, singer: subclass inherits base class attr.
human1 = Singer()
print()
human1.name = input("Enter your name: ")
human1.age = int(input("Enter your age: "))
human1.vocal_range = input("Enter your vocal range: ")

#singer sub_class method sing
human1.info()
human1.sing()

#object for case 3, dancer: subclass inherits base class attr.
human2 = Dancer()
print()
human2.name = input("Enter your name: ")
human2.age = int(input("Enter your age: "))
human2.dance_style = input("Enter your dance style: ")

human2.info()
human2.dance_move()
```

Performer:

```python
class Performer:  7 usages
    def __init__(self, name:str = None ,age:int = None):
        self.__name = name
        self.__age = age
#initial "None" null value for parameter to accept userinput

    @property #getter for dunder attr  7 usages
    def name(self):
        return self.__name
    @name.setter #setter for  dunder attr  5 usages
    def name(self, name:str):
        self.__name = name

    @property  5 usages
    def age(self):
        return self.__age
    @age.setter  3 usages
    def age(self, age:int):
        self.__age = age

    def info(self):  3 usages
        print(f"Name: {self.name} \n"
              f"Age: {self.age} years old")

#testing
if __name__ == '__main__':
    basePerson = Performer( name: 'John', age: 25)
    print(basePerson.name)
    print(basePerson.age)
```

Singer:

```python
from performer import Performer

class Singer(Performer):  2 usages
    def __init__(self, name = None, age = None, vocal_range:str=None):
        super().__init__(name,age) #inherits constructor name,age from Perfomer class
        self.__vocal_range = vocal_range #extra attr for child class

    @property  1 usage
    def vocal_range(self):
            return self.__vocal_range

    @vocal_range.setter  1 usage
    def vocal_range(self, vocal_range):
        self.__vocal_range = vocal_range


    def  sing(self):  1 usage
        print(f"{self.name} is singing with a vocal range of {self.__vocal_range}")
```

Dancer:

```python
from performer import Performer

class Dancer(Performer):  2 usages
    def __init__(self, name:str=None ,age:int= None, dance_style:str = None):
        super().__init__(name,age) #inherits the constructor from Parent class
        self.__dance_style = dance_style #extra attr of child class

    @property  1 usage
    def dance_style(self):
        return self.__dance_style

    @dance_style.setter  1 usage
    def dance_style(self, dance_style):
        self.__dance_style = dance_style

    def dance_move(self):  1 usage
        print(f"{self.name} is dancing {self.__dance_style}")
```